FraudDetectAI

Somyadeep Tarat (24BKT0104)
Aarav Gupta (24BCE0163)

AI-assisted banking fraud detection with verifiable funding evidence
Updated implementation report | 23 October 2026

1. Summary

FraudDetectAI combines a relational banking application, machine learning fraud scoring, transaction-network analysis and reproducible funding evidence. Customers own accounts and registered devices. A FastAPI service validates internal transfers, updates both balances, calculates risk, and stores predictions. Database triggers generate alerts and redacted audit records. A Streamlit interface supports banking operations and investigation.

The earlier submitted report described Funding Ambiguity Bounds Certificates (FABC) as a proposal. FABC is now implemented in the transfer workflow. A separate offline joint-funding prototype extends the research to collective source-funding bounds across final account holdings and recommends evidence queries that reduce decision uncertainty.

This report describes implemented code and saved evaluation results. It does not claim globally original mathematics, proven patent novelty, superior real-bank fraud detection, or production readiness. Earlier validation runs and their limitations are identified separately.

Report structure

Pages	Coverage
2-4	Scope, architecture and end-to-end transaction workflow
5-9	Complete schema dictionary and relationship diagrams
10-11	DBMS mechanisms, machine learning and graph analysis
12-13	FABC algorithm, worked examples and evidence protocol
14	Offline joint-funding and evidence-selection prototype
15-17	Recorded results, interface, deployment and conclusions

Project distribution

Docker Hub: https://hub.docker.com/r/sayantarat/frauddetectai

The repository Compose configuration names sayantarat/frauddetectai:latest for the backend. The model directory is mounted separately. Registry availability and image execution were not independently verified during report preparation.

2. Problem, objectives and implemented scope

2.1 Problem statement

Fraud is rare and often depends on account relationships or transaction history. An isolated transaction can appear normal while belonging to a suspicious network. Rapid receipt-and-forward behaviour can indicate pass-through activity, but legitimate merchants, payroll accounts and family collections can show the same pattern. Mixed funds also make unique source attribution unjustified.

2.2 Objectives

- Maintain valid banking state with relational keys, ownership checks and fixed-precision money.

- Score transfers with a fitted baseline model and separately recorded behavioural indicators.

- Explore graph features and evaluate historical features without target or future-label leakage.

- Calculate attainable recent-funding bounds and preserve the exact evidence for replay.

- Demonstrate atomicity, constraints, audit triggers, indexed queries and controlled concurrency.

- Investigate collective funding constraints and useful next evidence in an offline prototype.

2.3 Implementation status

Component	Current status
Banking API and dashboard	Implemented; same-currency internal transfers, registration, profiles, alert reviews and audits
Online inference	Fitted PaySim baseline adapter plus SQL behavioural features
FABC	Integrated with transfer, ledger, retained evidence, replay and a separate review queue
Graph analysis	Offline full-graph experiments and a separate label-free historical evaluation
Joint-funding research	Offline CLI and verifier; not integrated with live transfers
Deployment	Dockerfile and Compose configuration; PostgreSQL preferred, SQLite fallback

2.4 Financial and evidence boundaries

Supported transfers debit one known account and credit another in the same currency. Decimal/NUMERIC arithmetic is used for banking money. FABC treats the balance at the observation-window opening as old funds. Device and location metadata belong to the local application and are not PaySim attributes. Funding review is an investigation signal; it does not establish fraud or automatically freeze money.

3. System architecture

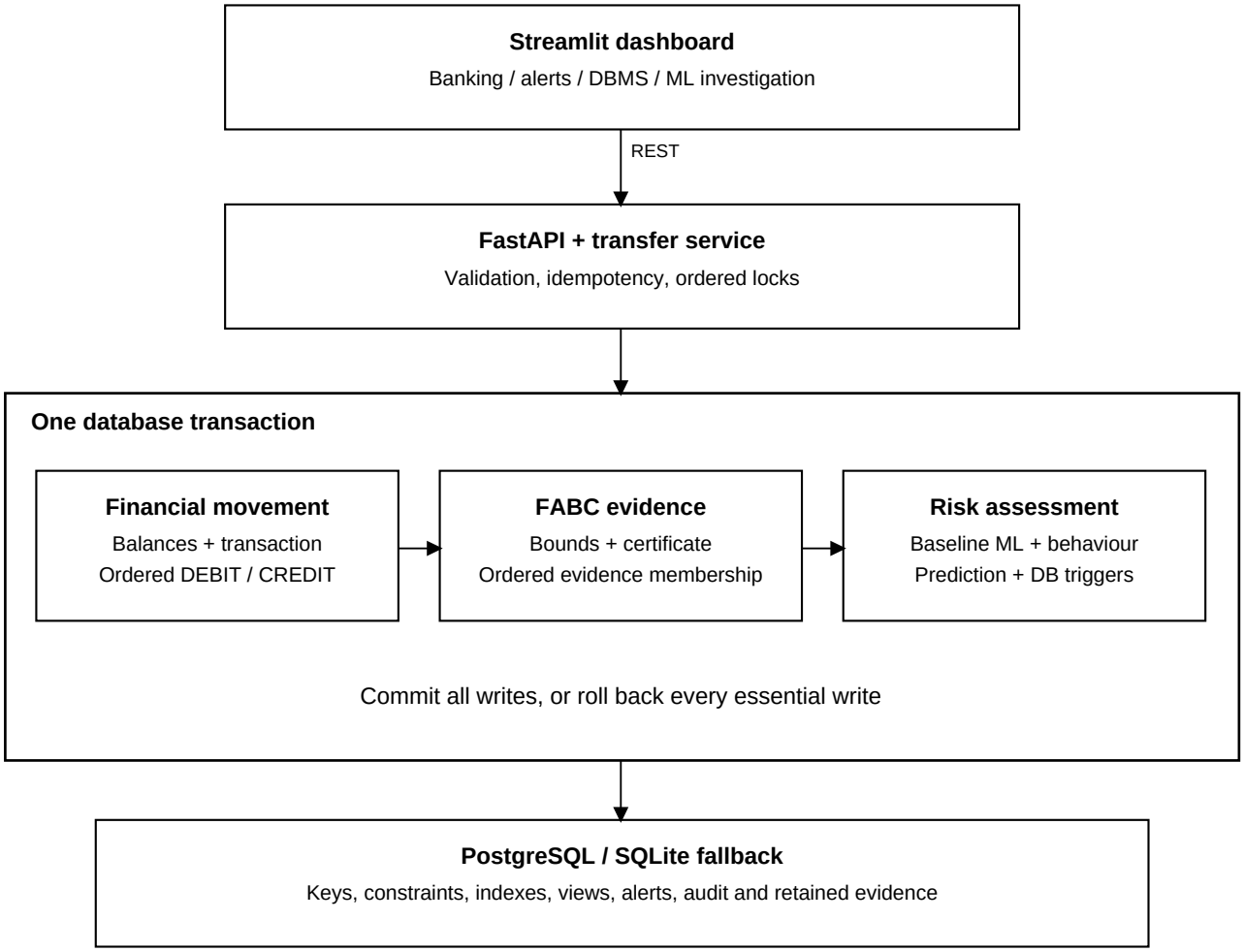

Figure 1. Implemented online architecture. FABC is inside the same transaction as the financial movement and prediction. PostgreSQL-specific objects have SQLite equivalents where supported.

3.1 Separate offline paths

```
PaySim CSV -> validation -> preprocessing -> transaction features
                         -> transaction-only model training
                         -> directed graph -> graph-enhanced training
                         -> evaluation / thresholds / explanations

Historical experiment -> earlier-step graph features -> RF comparison

Joint-funding JSON -> chronological capacity network -> min/max solves
                   -> certificate verifier -> evidence recommendation
```

The live adapter loads a baseline artifact. It does not execute the offline graph-enhanced model or the joint-funding optimizer during a transfer. Saved batch outputs feed the original analysis pages. The joint prototype reads and writes standalone JSON certificates.

3.2 Technologies

Python; FastAPI and Pydantic; SQLAlchemy and Alembic; PostgreSQL with a SQLite local fallback; pandas, scikit-learn, XGBoost and NetworkX; Streamlit; pytest; Docker and Docker Compose. Source modules are grouped under app/, src/fraudgraph/, alembic/, scripts/ and tests/.

4. End-to-end banking transaction workflow

1. Validate the request

Pydantic validates identifiers, finite positive amounts, precision and supported payment methods. The service checks customer status, account ownership, device ownership, sufficient funds, account status and matching currencies.

2. Begin and lock

The service begins a database transaction, locks the customer and takes account locks in globally sorted account-ID order. PostgreSQL uses row locks; SQLite uses writer-lock behaviour.

3. Handle retry identity

An optional globally unique idempotency key and canonical request digest identify the operation. A matching committed retry returns the existing transaction; a changed payload using the same key produces a conflict.

4. Record the movement

The service updates both balances, saves before/after snapshots and allocates one new sequence on each account. It inserts one DEBIT and one CREDIT ledger entry with server-recorded timestamps.

5. Produce funding evidence

The service finds the opening anchor and ordered source history, validates continuity, computes exact bounds or INCOMPLETE, and stores the certificate and ordered evidence membership.

6. Score and classify

Indexed SQL history supplies behavioural features; the fitted baseline supplies fraud_probability. risk_score = max(fraud_probability, 0.9 * behaviour_score). The database threshold controls classification and the prediction-insert alert trigger.

7. Commit or roll back

The prediction is inserted, alert and audit triggers run, and all essential writes commit together. An inference or certificate failure aborts balances, transaction, ledger, certificate, prediction and related writes.

Classification and review are distinct

The online classification is FRAUD when the model probability reaches the threshold, SUSPICIOUS when the combined heuristic risk reaches it, and NORMAL otherwise. These are application labels, not legal findings. FABC maintains a separate funding-review flag; it does not replace fraud_probability or silently reject the transfer.

5. Database schema: customers, accounts and devices

The following dictionary is generated from the current SQLAlchemy metadata. PK denotes primary key; FK denotes foreign key. String lengths and NUMERIC precision are preserved. Composite foreign keys are explained in section 8. JSON uses JSONB on PostgreSQL and JSON on SQLite. Defaults and cross-column checks are described in sections 8-9.

customers

Column	SQL type	Key / nullability
customer_id	VARCHAR(64)	PK; NOT NULL
name	VARCHAR(120)	NOT NULL
email	VARCHAR(254)	NOT NULL
phone	VARCHAR(30)	NULL allowed
status	VARCHAR(20)	NOT NULL
created_at	DATETIME	NOT NULL

accounts

Column	SQL type	Key / nullability
account_id	VARCHAR(64)	PK; NOT NULL
customer_id	VARCHAR(64)	NOT NULL; FK customers.customer_id
ledger_started_at	DATETIME	NULL allowed
ledger_seq	BIGINT	NOT NULL
account_type	VARCHAR(20)	NOT NULL
balance	NUMERIC(18, 2)	NOT NULL
currency	VARCHAR(3)	NOT NULL
status	VARCHAR(20)	NOT NULL
created_at	DATETIME	NOT NULL

devices

Column	SQL type	Key / nullability
device_id	VARCHAR(64)	PK; NOT NULL
customer_id	VARCHAR(64)	NOT NULL; FK customers.customer_id
device_type	VARCHAR(30)	NOT NULL
device_identifier	VARCHAR(120)	NOT NULL
ip_address	VARCHAR(45)	NULL allowed
first_seen	DATETIME	NOT NULL
last_seen	DATETIME	NOT NULL
trusted	BOOLEAN	NOT NULL

Additional uniqueness: customers.email; devices.device_identifier; accounts(account_id, customer_id); devices(device_id, customer_id). The composite owner keys support ownership-preserving references from transactions. ledger_started_at marks the beginning of reliable ledger observation for migrated accounts. ledger_seq is per account, not a global commit number.

6. Database schema: transfers and predictions

transactions

Column	SQL type	Key / nullability
transaction_id	VARCHAR(64)	PK; NOT NULL
account_id	VARCHAR(64)	NOT NULL; FK accounts.account_id
customer_id	VARCHAR(64)	NOT NULL; FK customers.customer_id, accounts.customer_id, devices.customer_id
destination_account_id	VARCHAR(64)	NOT NULL; FK accounts.account_id
idempotency_key	VARCHAR(64)	NULL allowed
request_digest	VARCHAR(64)	NULL allowed
amount	NUMERIC(18, 2)	NOT NULL
transaction_type	VARCHAR(20)	NOT NULL
payment_method	VARCHAR(20)	NOT NULL
transaction_timestamp	DATETIME	NOT NULL
location	VARCHAR(120)	NULL allowed
latitude	DOUBLE	NULL allowed
longitude	DOUBLE	NULL allowed
device_id	VARCHAR(64)	NULL allowed; FK devices.device_id
status	VARCHAR(20)	NOT NULL
balance_before	NUMERIC(18, 2)	NOT NULL
balance_after	NUMERIC(18, 2)	NOT NULL
destination_balance_before	NUMERIC(18, 2)	NOT NULL
destination_balance_after	NUMERIC(18, 2)	NOT NULL
created_at	DATETIME	NOT NULL

fraud_predictions

Column	SQL type	Key / nullability
prediction_id	VARCHAR(64)	PK; NOT NULL
transaction_id	VARCHAR(64)	NOT NULL; FK transactions.transaction_id
model_name	VARCHAR(120)	NOT NULL
model_version	VARCHAR(64)	NOT NULL
fraud_probability	DOUBLE	NOT NULL
behaviour_score	DOUBLE	NOT NULL
risk_score	DOUBLE	NOT NULL
classification	VARCHAR(20)	NOT NULL
features	JSON	NOT NULL
reasons	JSON	NOT NULL
predicted_at	DATETIME	NOT NULL

Additional uniqueness: transactions.idempotency_key (optional), fraud_predictions.transaction_id and fraud_predictions(prediction_id, transaction_id). Each committed service transfer has one prediction by application protocol; the schema alone enforces at most one. Monetary snapshots preserve event-time facts. The request digest detects conflicting retries.

7. Database schema: alerts, audit and policy

fraud_alerts

Column	SQL type	Key / nullability
alert_id	VARCHAR(64)	PK; NOT NULL
transaction_id	VARCHAR(64)	NOT NULL; FK transactions.transaction_id, fraud_predictions.transaction_id
prediction_id	VARCHAR(64)	NOT NULL; FK fraud_predictions.prediction_id
risk_score	DOUBLE	NOT NULL
reason	VARCHAR(500)	NOT NULL
alert_status	VARCHAR(20)	NOT NULL
created_at	DATETIME	NOT NULL
reviewed_at	DATETIME	NULL allowed
reviewed_by	VARCHAR(120)	NULL allowed

audit_logs

Column	SQL type	Key / nullability
audit_id	VARCHAR(64)	PK; NOT NULL
table_name	VARCHAR(64)	NOT NULL
record_id	VARCHAR(64)	NOT NULL
operation	VARCHAR(10)	NOT NULL
old_values	JSON	NULL allowed
new_values	JSON	NULL allowed
changed_at	DATETIME	NOT NULL
changed_by	VARCHAR(120)	NOT NULL

risk_settings

Column	SQL type	Key / nullability
settings_id	INTEGER	PK; NOT NULL
fraud_threshold	DOUBLE	NOT NULL

Additional uniqueness: fraud_alerts.prediction_id. The composite prediction/transaction foreign key prevents an alert from referring to a prediction for another transfer. Audit table_name and record_id are deliberately polymorphic references without entity foreign keys, allowing retained redacted history. risk_settings is a singleton restricted to settings_id = 1.

8. Database schema: ledger and funding evidence

ledger_entries

Column	SQL type	Key / nullability
account_id	VARCHAR(64)	PK; NOT NULL; FK accounts.account_id
seq	BIGINT	PK; NOT NULL
transaction_id	VARCHAR(64)	NOT NULL; FK transactions.transaction_id
side	VARCHAR(6)	NOT NULL
amount	NUMERIC(18, 2)	NOT NULL
balance_before	NUMERIC(18, 2)	NOT NULL
balance_after	NUMERIC(18, 2)	NOT NULL
recorded_at	DATETIME	NOT NULL

flow_certificates

Column	SQL type	Key / nullability
certificate_id	VARCHAR(64)	PK; NOT NULL
transaction_id	VARCHAR(64)	NOT NULL; FK transactions.transaction_id, ledger_entries.transaction_id
account_id	VARCHAR(64)	NOT NULL; FK accounts.account_id, ledger_entries.account_id
target_seq	BIGINT	NOT NULL; FK ledger_entries.seq
target_side	VARCHAR(6)	NOT NULL; FK ledger_entries.side
target_amount	NUMERIC(18, 2)	NOT NULL
status	VARCHAR(12)	NOT NULL
lower_amount	NUMERIC(18, 2)	NULL allowed
upper_amount	NUMERIC(18, 2)	NULL allowed
review	BOOLEAN	NOT NULL
algorithm_version	VARCHAR(64)	NOT NULL
policy_version	VARCHAR(64)	NOT NULL
payload	JSON	NOT NULL
evidence_digest	VARCHAR(64)	NOT NULL

certificate_evidence

Column	SQL type	Key / nullability
certificate_id	VARCHAR(64)	PK; NOT NULL; FK flow_certificates.certificate_id
account_id	VARCHAR(64)	PK; NOT NULL; FK ledger_entries.account_id
seq	BIGINT	PK; NOT NULL; FK ledger_entries.seq
ordinal	INTEGER	NOT NULL
role	VARCHAR(20)	NOT NULL

Unique keys: ledger_entries(transaction_id, side), ledger_entries(transaction_id, account_id, side, seq), flow_certificates.transaction_id, certificate_evidence(certificate_id, ordinal). FlowCertificate has a composite FK to the actual source DEBIT, and evidence has a composite FK to (account_id, seq). Opening balance, window, ordered records and related replay facts are retained in the versioned certificate payload; they are not separate scalar columns.

9. Entity relationships and cardinalities

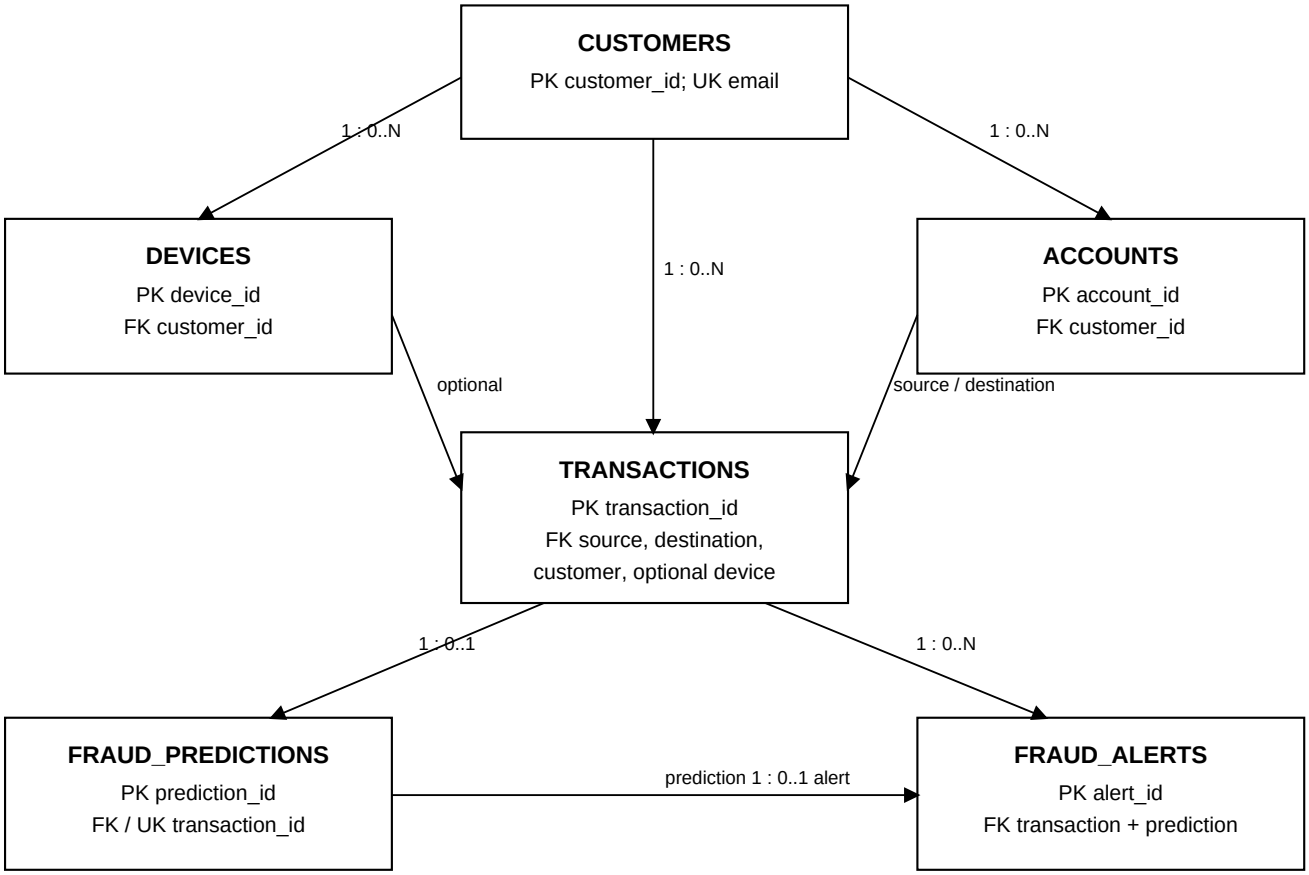

Figure 2. Banking relationships. Optional means a transfer may have no device. The source and destination are separate references to accounts. The service creates a prediction for every successful transfer.

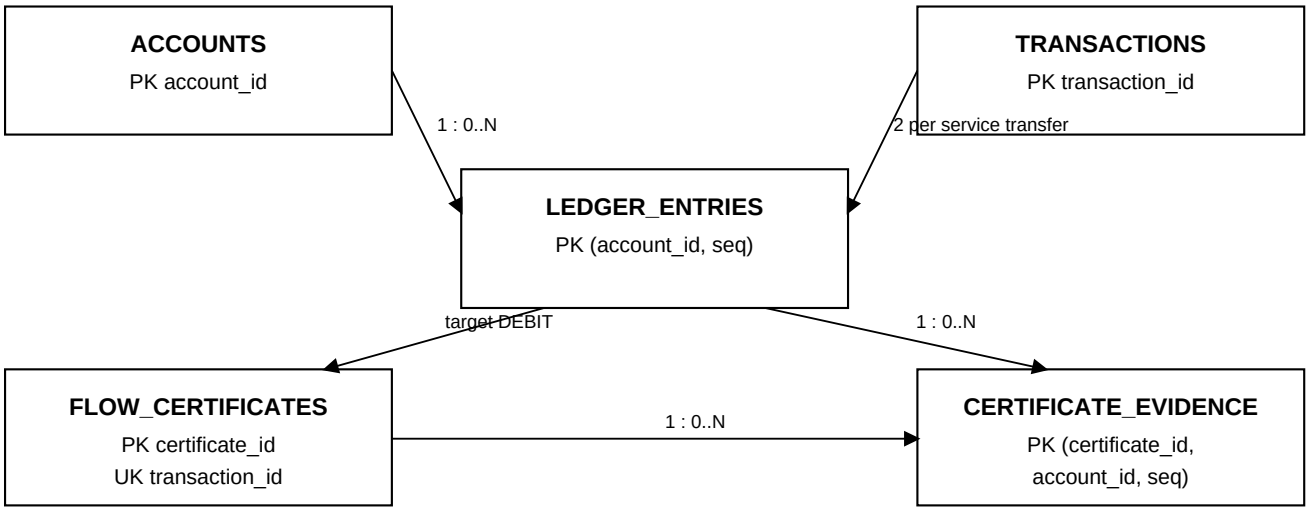

Figure 3. Evidence relationships. Each account and transaction can also own multiple ledger/certificate rows through their explicit FKs. A transfer has at most one certificate. Ledger entries can be reused by multiple certificates through the evidence relation. Two ledger entries per transfer is a service invariant, not a general cross-row CHECK guarantee.

10. DBMS design and integrity mechanisms

10.1 Constraints and normalization

Checks reject negative balances, nonpositive transfers, self-transfers, invalid states, invalid scores and invalid coordinates. Snapshot arithmetic requires source_after = source_before - amount and destination_after = destination_before + amount. Composite account/customer and device/customer FKs enforce ownership. Ledger checks enforce positive sequence and amount, nonnegative balances and side-specific arithmetic. A VALID certificate requires 0 <= lower <= upper <= target; INCOMPLETE requires NULL bounds and no review flag.

Customers, accounts, devices, predictions and alerts separate distinct facts. transaction.customer_id and alert.transaction_id are deliberate navigation denormalizations, constrained by composite FKs. Therefore the design should not be described as strictly pure 3NF throughout. Evidence membership resolves the many-to-many reuse of ledger entries by certificates; JSON captures versioned event evidence rather than replacing relational identity.

10.2 Triggers, views and SQL functions

Object	Purpose
prediction_alert trigger	After prediction insertion, creates an OPEN alert if persisted risk reaches the database threshold
Audit triggers	Capture redacted INSERT / UPDATE / DELETE history for customers, accounts, transfers, predictions and alerts
Evidence protection triggers	Reject updates/deletes to retained ledger/certificate evidence; protect certified financial facts
suspicious_transactions_view	Joins customer, transfer, prediction and optional alert
customer_risk_summary	Aggregated customer counts, amounts, risk and suspicious activity
flow_certificate_detail_view	Joins certificate, ordered evidence membership and ledger facts
PostgreSQL SQL functions	calculate_transaction_velocity and get_customer_transaction_stats

10.3 Indexes and migrations

Important indexes cover customer/time and account/time transaction history, prediction risk, alert status, ledger(account_id, recorded_at, seq), and certificate(review, status). PK/unique indexes support identity and retry checks. Use EXPLAIN rather than assuming an index improves tiny workloads. Alembic revisions create the base schema, database objects (0002), flow evidence (0003) and reporting/protection (0004).

10.4 ACID and retention

Atomicity comes from one transaction; constraints and service checks preserve consistency; ordered row locks coordinate cooperating PostgreSQL writers; committed records persist across connections. Reversals are new opposite-direction movements. The local FABC run did not verify PostgreSQL restart recovery or new PostgreSQL trigger execution. Direct monetary writers must follow the protocol.

11. Machine learning, graphs and explainability

11.1 Dataset and training pipeline

The offline pipeline uses PaySim columns such as step, type, amount, sender/receiver IDs, original/destination balances and isFraud. It validates and preprocesses data, engineers transaction features, trains Logistic Regression, Random Forest and XGBoost variants, compares models, tunes thresholds, assigns risk bands and saves explanations. Raw data and fitted artifacts are not committed; a compatible trusted baseline artifact is required for online transfers.

11.2 Transaction and graph features

Transaction features describe amount, type, balance changes, account draining and related arithmetic. A directed graph models sender_account to receiver_account. Node features include in/out degree, sent/received totals, transaction counts and averages. Edge features include pair transaction count, total amount and average amount. The original full-graph experiments also contain label-derived fraud rates and may include target/future information; their scores are unsuitable as evidence of leakage-free generalization.

A separate historical implementation uses no fraud-label aggregates. All transactions in one hourly PaySim step observe the same graph built from strictly earlier steps. The graph is updated only after that whole step is processed. The chronological comparison chooses thresholds on validation data and reports a later held-out test set. It does not establish ten-minute FABC quality because PaySim time is hourly.

11.3 Online risk contract

The baseline inference adapter constructs PaySim-compatible balance features from a live transfer and loads the existing fitted sklearn pipeline. It records model name and artifact SHA-256 version. The local mapping uses UTC hour-of-day plus one for step, unlike elapsed PaySim hours; this is an integration/domain limitation. Online graph features require a separate adapter and are not inferred automatically.

SQL behaviour features include recent transfer counts, historical average amount, deviation from average, previous-transfer interval, device trust and location changes. Weighted rules yield a bounded behaviour score. This is a heuristic, not a calibrated probability. ML probability, behaviour score and combined risk remain separately stored.

11.4 Outputs and evaluation

Offline evaluation includes precision, recall, F1, ROC-AUC, average precision and confusion matrices. Risk bands help prioritize review. Global feature importance and transaction-level rule explanations aid interpretation; they do not prove causation and are not an implemented SHAP attribution system. The dashboard supports comparisons and investigation of saved artifacts.

12. FABC: exact recent-funding bounds

FABC asks how much of a target outgoing payment must, and could, be funded by credits inside the source account's ten-minute observation window. It accounts for opening funds and every intervening debit. Bounds are exact under complete ordered evidence, one currency, no overdrafts and no unrecorded balance changes. They describe feasible allocations, not an observable unique funds identity.

12.1 Linear Decimal algorithm

Let B be the current balance, initially the opening balance. L and U are the minimum and maximum recent-funded amounts that can remain in the account. Initially L = U = 0. Before applying a debit, use the pre-debit B.

```
Credit c:
    B = B + c
    L = L + c
    U = U + c
Earlier debit d:
    L = max(0, L - d)
    U = min(U, B - d)
    B = B - d
Target debit x:
    lower_amount = max(0, x - (B - L))
    upper_amount = min(x, U)
    lower_share = lower_amount / x
    upper_share = upper_amount / x
```

Spending recent funds first at prior debits minimizes the recent balance left; spending old funds first maximizes it. These feasible extremes propagate through the ordered history. The scan uses O(n) arithmetic and O(1) working state; retaining the full evidence requires O(n) storage. Marginal bounds for different payments cannot be summed as though their endpoints were jointly attainable.

12.2 Worked example: unavoidable recent funding

Event (INR)	Balance B	Recent balance [L, U]
Opening balance 1,000	1,000	[0, 0]
Credit 4,000 from A	5,000	[4,000, 4,000]
Credit 3,000 from B	8,000	[7,000, 7,000]
Credit 2,000 from C	10,000	[9,000, 9,000]
Earlier debit 2,000	8,000	[7,000, 8,000]
Target debit 7,500	Pre-target: 8,000	Target interval: [6,500, 7,500]

The minimum share is 86.67%, with three incoming counterparties, so the fixed demo policy requests review. With opening funds of INR 100,000, recent receipts of INR 9,000 and a target of INR 7,500, the interval instead becomes [0, 7,500]: old funds alone could cover the entire payment. Neither example determines fraud.

13. Certificate persistence, replay and failure handling

13.1 Evidence selection and completeness

The service reads the source prefix before the target DEBIT. A ten-minute inclusive start identifies recent credits; a preceding ledger entry provides an opening anchor, or a known opening state is used when valid. Per-account sequences and before/after arithmetic are checked for continuity. Server-recorded nondecreasing timestamps govern observation, so caller timestamps cannot reorder the funding evidence.

The scan cap is 10,000 history entries. Missing anchors, balance/sequence gaps, an unobserved legacy period or a capped history produce INCOMPLETE, NULL bounds and review=false. Migration preserves old transfers without inventing retrospective certificates. A migrated account can remain incomplete until a full observation window is available.

13.2 Stored certificate

A certificate stores target identity, source account, target sequence/amount, status, bounds, review flag, versions, a canonical payload and SHA-256 digest. certificate_evidence records ordered membership with OPENING_ANCHOR, HISTORY and TARGET roles. Money is represented as fixed two-decimal strings and timestamps as UTC in the payload. Retention triggers reject evidence mutation; certified transfer financial facts are protected.

13.3 Replay

Replay checks the certificate identity, versions, digest, membership ordering, live ledger facts and recomputed bounds/results. A matching result shows agreement with retained inputs and implementation. It does not authenticate the original source data or defeat a privileged administrator who rewrites the data and digest together. Backups, restricted database roles and external trust controls remain necessary.

13.4 Separate analyst review

The fixed funding policy requests review when lower_share is at least 0.80 and at least three distinct incoming accounts appear in the ten-minute window. These are demo parameters, not regulatory thresholds. The Banking page provides certificate inspection, replay and a funding-review queue. An incomplete result is visible uncertainty, not a numerical zero.

13.5 API and transaction guarantees

```
POST /transactions  (optional idempotency_key)
GET /transactions/{id}/flow-certificate
GET /transactions/{id}/flow-certificate/replay
GET /fraud/flow-reviews
```

Certificates are generated before ordinary prediction insertion inside the same transaction. Failure after certificate writes must roll back all associated movement and prediction state. A transfer predating the migration has no certificate and inspection returns 404. Two-entry conservation relies on the transfer service and cooperating writers; a row CHECK cannot enforce the full cross-row protocol.

14. Offline joint-funding research prototype

14.1 Collective bounds

The joint prototype builds a chronological capacity network for one source cohort, one currency and complete ordered internal movements plus supported external credits. Opening accounts and external credits carry independently supported source-attribution intervals. The optimizer minimizes and maximizes the cohort amount held across selected final accounts. Internal transfers move capacity; they do not inject a new source.

Example: INR 10,000 of selected-source funds mixes with INR 10,000 of other funds and ends in two accounts holding INR 10,000 each. Each account separately can contain [0, 10,000] from the source, but the two together must hold exactly INR 10,000. Joint conservation captures the dependency lost by adding independent marginal intervals. Targets are final holdings, not arbitrary serial transfers.

14.2 Certificate and independent verification

Monetary values use integer minor units. Output includes lower/upper bounds, a threshold condition (MET, NOT_MET or UNRESOLVED), endpoint flow witnesses, version/snapshot metadata and a digest. A separate verifier checks capacity, conservation and optimality using the absence of negative residual cycles, without rerunning the optimizer. It still depends on trustworthy input evidence.

14.3 Decision-directed evidence selection

Each query addresses an obtainable boundary record with positive acquisition cost. Its possible integer outcome intervals must partition the current boundary interval. The prototype evaluates outcomes, ranks guaranteed decision-ambiguity reduction per cost, and can apply a chosen outcome to a copy of the ledger before fully recomputing bounds. Selection is deterministic and greedy; useful combinations of individually unhelpful queries may be missed.

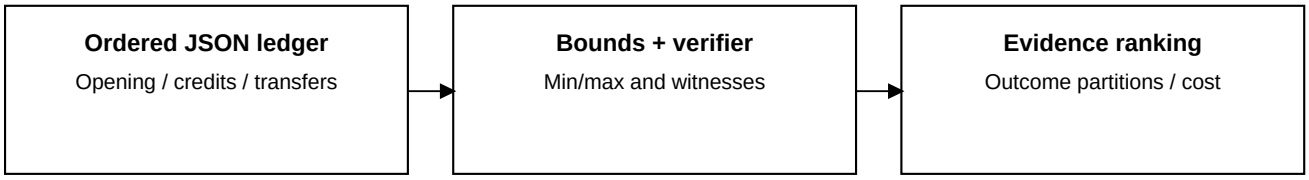

Figure 4. Offline prototype pipeline. It is not invoked by the transfer API.

14.4 Explicit scope limits

There is no bank connector, learned response distribution, incremental solver cache or globally optimal multi-query planner. External withdrawals/re-entry, FX, fees, overdrafts and overlapping source cohorts are outside scope. A query cannot treat an unobservable internal fungible allocation as obtainable evidence. The current synthetic query experiment ties a simple reachability baseline; no superiority claim is supported.

15. Recorded evaluation and measured limitations

The following are saved repository results, not experiments rerun for this report. Different suites correspond to different implementation stages.

15.1 FABC and database evaluation

Measure	Recorded result
FABC implementation-stage tests	122 passed, 1 PostgreSQL-dependent skip
Synthetic scenario study	1,200 targets; FABC precision 39.25%, recall 42.00%; 130 false alerts per 1,000 benign targets
SQLite replay benchmark	100 transfers, 100 persisted certificates, all replayed
Ledger/certificate workload latency	p50 / p95 / p99: 20.20 / 22.01 / 22.46 ms
Control latency	17.11 / 18.29 / 18.53 ms; disables both ledger and certificate writes
Sequential throughput	49.05 transfers/s with evidence; 57.75 transfers/s control
Indexed history plan	Used idx_ledger_account_recorded_seq

This is a warm-model sequential local SQLite workload, not PostgreSQL lock-wait or concurrent throughput validation. Combined ledger/certificate overhead is measured, not certificate-only overhead. Synthetic benign collections can resemble suspicious histories; haircut achieved higher recall and average precision in this constructed study. The ML comparison on synthetic snapshots is a domain-mismatch stress test.

15.2 Historical PaySim comparison

A seeded 2% sample contained 127,171 rows: 117,936 train (91 fraud), 5,401 validation (34 fraud), and 3,834 test (38 fraud). Validation starts at step 404; test at 564. Thresholds are selected on validation only.

Model	Test TP / FN / FP	Precision / recall / F1
Transaction Random Forest	38 / 0 / 0	1.000 / 1.000 / 1.000
Historical-graph Random Forest	36 / 2 / 0	1.000 / 0.9474 / 0.9730

The small sampled test does not establish generalization or a graph improvement. Both methods had 3,796 true negatives. The historical graph is sampled rather than a complete banking network.

15.3 Joint prototype validation

The recorded full suite passed 394 tests with one skip; 272 new tests included 250 generated ledgers matching exhaustive integer allocation. Joint analysis including verification had p50/p95 0.4281/0.4964 ms. In 100 deliberately simple one-query cases, proposed selection and reachability filtering resolved 100/100, widest-boundary 0/100, and seeded random 40/100. These are proof-of-concept results, not fraud accuracy or real analyst workload.

16. Interface, deployment and project demonstration

16.1 User-facing modules and API

Banking provides customer/account/device registration, transfers, history filters, stored predictions, alert review, customer profiles and FABC inspection/replay. DBMS Demonstration exposes objects, audit history, query plans, rollback and concurrency examples. Original Overview, Fraud Detection, Graph Explorer, Model Performance and Explainability pages display batch outputs.

API groups include /customers, /accounts, /devices, /transactions, /fraud/alerts, /fraud/statistics, /audit-logs and /database/overview, alongside the certificate routes. Alert reviews update workflow status without changing certified financial facts. Development demonstration routes are omitted when APP_ENV=production; this does not add authentication.

16.2 Docker and local setup

Docker Hub repository: https://hub.docker.com/r/sayantarat/frauddetectai. The Compose backend image is sayantarat/frauddetectai:latest and also supports a local build. PostgreSQL uses a named persistent volume. Both backend and database ports bind to localhost.

```
docker pull sayantarat/frauddetectai:latest
# From the project directory, with a compatible model in ./models:
docker compose up -d postgres
docker compose up -d backend
# Alternative: docker compose up --build -d

# Host Python environment:
export PYTHONPATH=src:.
# Set DATABASE_URL for the selected database.
alembic upgrade head
python scripts/seed_database.py
uvicorn app.main:app --reload
# Second terminal:
streamlit run app/streamlit_app.py
```

Compose starts migrations before Uvicorn and mounts ./models read-only. A raw standalone docker run is insufficient without a configured database and fitted model. For SQLite, use DATABASE_URL=sqlite:///data/banking.db. PostgreSQL is required for row locks, JSONB and SQL-function demonstrations. The image was not pulled or executed as part of preparing this report.

16.3 Concise presentation sequence

- Show the customer/account relationship and submit a valid transfer.

- Open the stored probability, behaviour score and alert; explain their distinction.

- Inspect the FABC interval, evidence sequence and replay result.

- Demonstrate rollback after an injected essential failure, then concurrent debits.

- Run the offline joint-funding example and compare individual versus collective bounds.

- End with measured limitations rather than claiming production detection superiority.

17. Contribution, limitations and conclusion

17.1 Defensible contribution

The implemented contribution is an uncertainty-aware banking evidence workflow: attainable funding bounds, a validated ordered ledger prefix, atomic persistence with a transfer, retained evidence and replay. The offline extension investigates joint source-cohort holdings and cost-aware evidence acquisition. The mathematical constituents, database locks and hashes should not individually be described as inventions. Global originality and patent novelty remain unresolved; no new patent assessment was performed for this report.

17.2 Remaining limitations and future work

- The course application has no implemented authentication; production authorization, secrets management and operational controls remain necessary.

- The pre-FABC validation records PostgreSQL checks, but the later FABC/joint runs do not establish execution of the new PostgreSQL triggers, row-lock performance or server-restart recovery.

- All movement writers must obey account locks and ledger conservation. Administrator bypass and arbitrary direct balance updates are outside the evidence guarantee.

- PaySim and designed synthetic scenarios do not validate real-bank behaviour, minute-level funding policies, calibrated online probabilities or broad generalization.

- The joint prototype remains offline and restricted to its complete-ledger input contract. Stronger connected-path benchmarks, realistic evidence costs and query complementarity remain future research.

- Incremental window-expiry maintenance, learned evidence value, online graph adaptation, GNNs and SHAP explanations are not implemented contributions of this report.

17.3 Conclusion

FraudDetectAI demonstrates how banking transaction integrity, AI risk assessment and funding explanations can coexist in a single project. Its strongest result is reproducible, explicitly bounded evidence rather than a claim that suspicious funding proves fraud. The application integrates FABC with the transfer transaction; the offline prototype extends the investigation to collective account holdings. Recorded tests and benchmarks support feasibility within stated assumptions, while showing where detection and deployment claims remain limited.

18. Sources and reproducibility

Primary implementation sources: app/db/models.py; app/db/objects.py; app/api/routes.py; app/services/transactions.py, inference.py, features.py, flow_bounds.py, flow_evidence.py and joint_funding.py; alembic/versions/; src/fraudgraph/features/historical_graph.py; docker-compose.yml; Dockerfile.

Recorded evidence: docs/COMPLETION_AUDIT.md; docs/VALIDATION.md (earlier stage); docs/JOINT_FUNDING_PROTOTYPE.md; docs/evaluation/flow_evaluation.json, flow_database_benchmark.json, historical_graph_evaluation.json and joint_funding_evaluation.json. Reproduction entry points: scripts/evaluate_flow.py, benchmark_flow_database.py, evaluate_historical_graph.py, evaluate_joint_funding.py and run_joint_funding.py.

Structural reference: supplied 24BKT0104_VL2026270101632_CS1.pdf. Its proposed architecture/schema are superseded here by the current implementation. The author names and registration numbers are retained. Docker repository URL was supplied by the project owner. This report updates implementation facts without repeating the earlier report's unverified legal or prior-art assertions.